

Chapter 4: Hardening & Secure Configuration

CompTIA Security+ Study Slides

Walailak University — Assist. Prof. Dr. CJ

Learning Objectives

- Define the concept of **system hardening**
- Identify unnecessary services, applications, and protocols
- Explain **patch management** and **baseline security**
- Describe **group policies**, **file systems**, and **update management**
- Apply secure configuration best practices to OS and servers

What is Hardening?

Definition:

Hardening is the process of **securing an operating system or device** by reducing its attack surface —
removing vulnerabilities, disabling unused functions, and enforcing strict configurations.

 **Goal:** minimize exposure, mitigate risk, and ensure system integrity.

Principles of Hardening

- Remove unnecessary software, services, and user accounts
- Apply the principle of **least functionality**
- Regularly patch and update all components
- Implement secure configuration baselines
- Use auditing and logging to detect deviations

Security cannot be guaranteed — but risk can be minimized.

Least Functionality

Concept: Each system should run **only the services and applications required** for its role.

Examples:

- Disable Bluetooth, telnet, FTP on servers not using them
- Turn off auto-run on removable drives
- Use minimal OS images for servers or appliances

 *The fewer components running, the fewer vulnerabilities exist.*

Removing Unnecessary Applications

- Avoid software that introduces vulnerabilities or complexity
- Periodically review installed applications
- Use **secure baseline images** for new deployments
- Example: Microsoft SCCM (System Center Configuration Manager) can automate baselines

Unnecessary Services

- Disable all unneeded network services (e.g., SMBv1, Telnet, SNMPv1)
- Harden startup processes and system daemons
- Verify using:
 - `services.msc` (Windows)
 - `systemctl` or `chkconfig` (Linux)
- Use firewall rules to restrict port exposure

Trusted Operating Systems (TOS)

A **Trusted OS** is one that meets specific security certification standards and implements multi-level security.

Examples:

- Windows 10 / 11 Enterprise
- macOS 10.6 or later
- Red Hat Enterprise Server, TrustedBSD

Features:

- Strong authentication
- Mandatory access control
- Audit capabilities and kernel integrity checking

Patch Management

Patch: A software update designed to fix bugs or security vulnerabilities.

Categories:

Type	Description
Security Update	Addresses security-related vulnerabilities
Critical Update	Resolves high-impact non-security bugs
Service Pack	Collection of patches and enhancements
Driver Update	Fixes or improves hardware compatibility



Patch Management Process

1. **Planning** — Identify systems and prioritize critical assets
 2. **Testing** — Verify patch compatibility in a sandbox
 3. **Implementation** — Apply via manual or automated deployment
 4. **Verification** — Confirm patch success and monitor logs
 5. **Auditing** — Document and review compliance
- 💡 Centralized patch servers (WSUS, SCCM, Ansible, JAMF) help ensure consistency.

Hotfix vs. Patch

- **Hotfix:** Immediate fix for a single urgent issue
- **Patch:** Generic software correction (may include multiple fixes)
- Today, both terms are often used interchangeably

 Always **test before deployment** to prevent operational disruption.



Group Policy (Windows)

Group Policy: Centralized management of user and computer configurations in Active Directory.

- Access via `gpedit.msc` or Group Policy Management Console (GPMC)
- Common policy areas:
 - Password complexity and expiration
 - Account lockout and authentication methods
 - Software and application restrictions
 - Windows Update, firewall, and auditing

Group Policies enforce consistent security across all users and machines.

Security Templates

- A **Security Template** is a predefined collection of policies applied to systems.
- Can be imported or customized to set:
 - Password length and age
 - User rights assignments
 - Event auditing and permissions

 Helps administrators **standardize** hardening procedures.

Baseline Security Configuration

Baseline: A snapshot of a secure system configuration used for comparison and auditing.

Purpose:

- Identify configuration drift or deviations
- Maintain consistency across environments
- Serve as rollback or recovery reference

Tools:

- Microsoft Security Compliance Toolkit (SCT)
- CIS Benchmarks, DISA STIGs
- Automated configuration scanning (OpenSCAP, Nessus, Lynis)

File Systems and Security

File System	Platform	Notes
NTFS	Windows	Supports ACLs, encryption (EFS), journaling
FAT32	Windows (legacy)	No permissions or logging; not secure
ext4	Linux	Journaling, ACLs, quotas
APFS	macOS	Modern, supports encryption and snapshots

Recommendation:

Always choose a file system that supports **permissions, auditing, and encryption**.

Hard Drive Maintenance

Regular maintenance extends drive life and integrity.

- Use **Disk Cleanup** to remove temp files
- Run **file system checks** (chkdsk, fsck)
- **Defragment** HDDs (not SSDs)
- Schedule **backups** and verify restoration procedures
- Practice **data recovery drills**



Account & Password Policies

- Enforce strong passwords (minimum 14+ characters)
- Enable **multi-factor authentication (MFA)**
- Use **account lockout policies** to deter brute-force attacks
- Disable or remove **default and inactive accounts**
- Apply **least privilege** and regular permission reviews

System and Application Hardening Checklist

- ✓ Disable unnecessary ports and protocols
- ✓ Install antivirus and endpoint protection
- ✓ Keep systems fully patched
- ✓ Enable firewalls and IDS/IPS
- ✓ Encrypt sensitive files and backups
- ✓ Enforce password and access controls
- ✓ Monitor logs and configure alerts

Change and Configuration Management

- Use **version control** for configuration files (Git, Ansible)
- Maintain change approval workflows (CAB)
- Keep inventory of hardware/software baselines
- Document all security changes
- Automate compliance checks when possible

Automation Tools

- **Windows:** PowerShell DSC, SCCM, WSUS
- **Linux:** Ansible, Puppet, Chef, SaltStack
- **macOS:** JAMF, Munki
- Automation ensures repeatable, secure configurations across environments.

Key Takeaways

- System hardening reduces the attack surface and enforces least privilege
- Patch management ensures systems remain protected from known exploits
- Baselines and policies provide consistency and compliance
- Automation enhances security reliability and scalability

End of Chapter 4

Next: [Chapter 5 — Virtualization & Cloud Security](#)